Milestone 03: Kinematics Analysis Report

Aly Serry

November 12, 2025

1 Kinematics Analysis: Steps & Explanation

This report details the steps taken to implement and validate the forward/inverse velocity and acceleration kinematics for the UR5e robotic arm. The analysis was performed in Python using the `ikpy` library, and all calculations were validated against the live MuJoCo simulation.

1.1 Kinematic Chain Setup

Before any calculations, a correct mathematical model of the robot was established.

- **URDF Generation:** A static `ur5e.urdf` file was generated from the official `ur_description` xacro files. This "flattens" the robot's blueprints into a single file that `ikpy` can parse, which we store in our `stacking_robot` package.
- **Chain Loading:** The `ikpy.chain.Chain.from_urdf_file()` function was used to load this URDF and create a "chain" object.
- **Active Link Masking:** A diagnostic script (`explore_chain.py`) revealed that the full `ikpy` chain contains **9 links**, but only **6 are moving (`revolute`) joints**. The rest are fixed. To get a correct 6x6 Jacobian, we must pass a custom `active_links_mask` to the loader to tell it which joints to use:

```
[False, False, True, True, True, True, True, True, False]
```

1.2 Velocity & Acceleration Kinematics Implementation

The following steps were implemented in our `velocity_kinematics.py` script.

1.2.1 Forward Velocity Kinematics ($v = J \cdot \dot{q}$)

The primary goal was to find the end-effector's 6D velocity (v) given the 6 joint velocities ($\dot{q}$). This requires the **Jacobian matrix (J)**.

Since `ikpy` does not provide a simple public function for the Jacobian, we created a custom `get_jacobian` function based on the standard robotics formula for revolute joints:

- The i-th column of the Jacobian is a 6D vector $\begin{bmatrix} J_v \\ J_w \end{bmatrix}$.
- J_v **(Linear Velocity):** $z_i \times (p_e - p_i)$
- J_w **(Angular Velocity):** z_i

Our script implements this by:

1. Calling `chain.forward_kinematics(full_kinematics=True)` to get the transformation matrices (T) for all 9 links.
2. Extracting the end-effector's position (p_e) from the final link's matrix.

3. Looping through our 6 active joint indices (2-7).
4. For each joint, extracting its origin (p_i) and rotation axis (z_i) from its matrix.
5. Calculating J_v and J_w and assembling the final 6x6 Jacobian (J).
6. Finally, calculating the end-effector velocity with $v = J \cdot \dot{q}$.

1.2.2 Inverse Velocity Kinematics ($\dot{q} = J^{-1} \cdot v$)

The script also implements the inverse calculation to find the required joint velocities ($\dot{q}$) for a desired end-effector velocity (v).

1. It takes the Jacobian (J) from the previous step.
2. It uses **`np.linalg.pinv(J)`** (the Moore-Penrose pseudo-inverse) to find J^{-1}. This is more robust than a standard inverse, as it prevents crashes if the robot is in a singular configuration.
3. It calculates the joint velocities: $\dot{q}_{calculated} = J^{-1} \cdot v$.
4. The script performs a self-check by comparing $\dot{q}_{calculated}$ to the original $\dot{q}$, confirming the math is sound.

1.2.3 Acceleration Kinematics ($a = J \cdot \ddot{q} + \dot{J} \cdot \dot{q}$)

To find the end-effector's acceleration (a), we require the **Jacobian Derivative** ($\dot{J}$).

We created a `get_jacobian_derivative` function that uses the **finite difference method** for numerical approximation:

1. It gets the current Jacobian $J(q)$.
2. It calculates the joint angles a tiny step forward in time: $q_{next} = q + \dot{q} \cdot \Delta t$.
3. It calls `get_jacobian()` again to get $J(q_{next})$.
4. It calculates the derivative: $\dot{J} \approx \frac{J(q_{next}) - J(q)}{\Delta t}$.
5. Finally, it calculates the full acceleration: $a = J \cdot \ddot{q} + \dot{J} \cdot \dot{q}$.

1.3 Simulation Validation & Testing

The kinematic equations were validated against the live MuJoCo simulation, as shown in the submitted videos.

1.3.1 Position Kinematics Test (Req #5)

We first validated our position model to ensure the URDF and MuJoCo models matched.

- **Method:** We ran `test_kinematics.py`, which subscribes to `/ur5e/joint_states`, reads the robot's *actual* position from the sim, and runs it through our `ikpy` forward kinematics.
- **Result:** The test passed successfully. The small, expected error of **8.41mm** confirms our kinematic chain is correct. This minor discrepancy is due to using two different models: our `ikpy` chain (from `ur_description`) and the MuJoCo model (from `mujoco_menagerie`).

```
--- KINEMATICS TEST ---
Calculated (Expected) Position: [ 0.1333 -0.4919  0.4879]

[INFO] [1762915991.782876854] [kinematics_tester]: Waiting for 'ur5e/joint_states' from simulation
    ...
[INFO] [1762915991.815548344] [kinematics_tester]: Received joint states from MuJoCo!
Simulated (Actual) Position:  [ 0.1338 -0.4936  0.4797]

--- RESULTS ---
Position Error (Difference): 0.008410 meters
TEST PASSED: Kinematic models match!
```

1.3.2 Velocity Kinematics Test (Req #3)

We then validated our velocity model using a live-tracking script.

- **Method:** We ran `validate_velocity_kinematics.py`. This script subscribes to `/ur5e/joint_sta` and, as we move the robot with our GUI, it continuously runs our $v = J \cdot \dot{q}$ calculation using the *live* q and $\dot{q}$ data from the simulator.
- **Result:** The validation was successful. The outputs below (and in the video) show the script's behavior.

1. At Rest: When the robot is still, the script correctly reads zero joint velocities and calculates zero end-effector velocity.

```
--- LIVE KINEMATICS VALIDATION -----
  q (actual): [-1.57  -1.562  1.578 -1.567 -1.57  -0.   ]
q_dot (actual): [-0. -0.  0.  0. -0. -0.]
  v (expected): [-0.  0. -0.  0.  0. -0.]
```

2. In Motion: As the base joint is moved via the GUI, the script tracks the motion in real-time. It reads the high-speed joint velocity (`q_dot (actual)`) and correctly calculates the corresponding non-zero end-effector velocity (`v (expected)`).

```
--- LIVE KINEMATICS VALIDATION -----
  q (actual): [-0.938 -1.549  1.577 -1.568 -1.57   0.   ]
q_dot (actual): [10.149  0.181 -0.008 -0.01  -0.002  0.   ]
  v (expected): [ 3.308  4.033 -0.085  0.131  0.098 10.149]
```

This live test confirms our `get_jacobian` function is correct and successfully translates joint-space velocity to task-space velocity.